

SystemView kayıt analizi

Üç senaryoda task davranışı, zamanlama ve USART6 kayıt trafiği. İnceleme, dosyalardaki Start/Stop bölümlerini ayrı ayrı değerlendirir ve bulguları mevcut uygulama koduyla ilişkilendirir.

93,60 s

Toplam incelenen süre

298.180

Benzersiz trace olayı

0

Overflow olayı

Asıl değişim: buton görevinin bekleme süresi

5 ms yük eklendiğinde ButtonTask'ın en uzun hazırdan çalışmaya geçiş süresi 0,144 ms'den 5,327 ms'ye çıktı. Yüklü bölümde 60 örneğin 24'ü 1 ms'den fazla bekledi. En kötü örnekte TelemetryTask çalışırken buton kesmesi geldi; ButtonTask hazır oldu, ancak daha yüksek öncelikli telemetri görevi bitene kadar çalışamadı.

Aktif çalışma bölümü	10 ms telemetri	+ 5 ms yük
TelemetryTask bağlam payı	%1,67	%52,95
Idle bağlam payı	%96,05	%44,70
ButtonTask en uzun bekleme	0,144 ms	5,327 ms
Telemetri ortalama üretim aralığı	10,000 ms	10,000 ms

921600 baud, bu kayıt yükünde yeterli görünüyor

En yoğun tam bir saniyede üretilen trace verisi 24.351 bayt. Bu, 921600 baud / 8N1 hattın teorik kapasitesinin %26,42'si. Dosyalarda Overflow olayı ya da ayrıştırma bozukluğu yok. Bu bulgu, incelenen üç kayıt için geçerli; kısa süreli tampon taşmalarına karşı bütün olası yükler için garanti değildir.

Ölçüm sınırı: Buradaki bekleme, **task READY** → **RUN** aralığıdır; butondan UART/GUI'ye toplam yanıt süresi değildir. Bağlam yüzdeleri de izlenen olaylara dayanır; izlenmeyen kesmeler ve kayıt maliyeti mutlak CPU yükünden ayrılamaz.

Dosyalar ve doğru karşılaştırma

Dosyalar birikimli kaydedilmiş. İkinci dosyanın ikili veri başlangıcı ilk dosyayla, üçüncü dosyanın başlangıcı da ikinci dosyayla birebir aynı. Dolayısıyla dosyaların tamamını üç bağımsız test gibi karşılaştırmak önceki senaryoları tekrar sayar.

Dosya / yeni bölüm	Dosya boyutu	Yeni bölüm	Yeni olay
No_Telemetry.SVDat	303.777 B	29,841 s	67.259
10ms_telemetry.SVDat	955.812 B	32,787 s	118.997
10ms_5msLoad.SVDat	1.566.376 B	30,973 s	111.924

Dosya adları senaryo etiketi olarak kullanıldı. Periyodun yaklaşık 10 ms olması ve üçüncü bölümde telemetri çalışmasının yaklaşık 5,3 ms sürmesi bu adlarla uyumlu. CFG komutunun içeriği ve load_us alanı bu trace dosyalarından doğrudan okunmadı.

Aktif pencere neden ayrı?

İki telemetri kaydında test öncesinde ve sonrasında Idle ağırlıklı süreler bulunuyor. Yük karşılaştırmasında ilk ve son TelemetryTask kuyruk gönderim olayı arasındaki bölüm kullanıldı. Sonraki tüm grafiklerde zaman, ilgili Start olayına göre gösterilir.

Senaryo	Tam bölüm	Aktif pencere	Pencere süresi
Telemetri kapalı	29,841 s	0,000 - 29,841 s	29,841 s
10 ms telemetri	32,787 s	5,048 - 28,238 s	23,190 s
10 ms + 5 ms yük	30,973 s	6,019 - 27,639 s	21,620 s

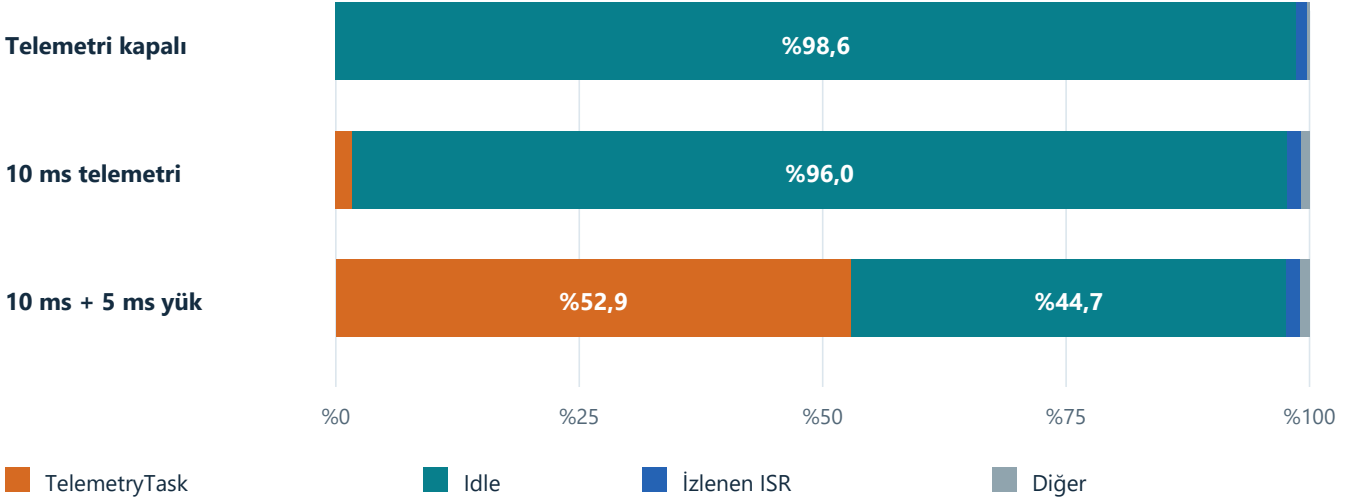
Çözümleme ve doğrulama

İkili akış, projedeki SEGGER paket koduna göre çözüldü. INIT olayında timestamp frekansı 216 MHz; zamanlar paketlerdeki hedef zaman farklarından hesaplandı. ISR giriş/çıkışları ve task geçişleri üzerinden bağlam süreleri toplandı. ISR'den scheduler'a çıkış ile sonraki task başlangıcı arası ayrı tutuldu.

Tüm paketler yeniden kodlandığında kaynak baytlarıyla birebir eşleşti. 298.180 benzersiz olayda ayrıştırma hatası, eşleşmeyen ISR çıkışı veya yinelenen bekleyen READY kaydı yok. Her bölümde bağlam süreleri bölüm süresine eşit. TIM6 sayısı da saniyede yaklaşık 1000 olay ile zaman ölçeğini destekliyor.

Bu çalışma özel bir çözücü ile üretildi; SEGGER'in CSV dışı aktarımı değildir. Doğrulamalar dosyanın yapısal tutarlılığını gösterir. Trace dışında kalan süreleri ve uygulama paketlerinin başarı durumunu ayrıca ölçmez.

İşlemci zamanının paylaşımı



İzlenen bağlam	Telemetri kapalı	10 ms telemetri	+ 5 ms yük
Idle	%98,597	%96,046	%44,702
TelemetryTask	%0,000	%1,669	%52,947
UartTxTask	%0,237	%0,779	%0,783
ButtonTask	%0,000	%0,037	%0,038
ISR	%1,119	%1,380	%1,374
Scheduler	%0,000	%0,090	%0,157
Bilinmeyen	%0,047	%0,000	%0,000

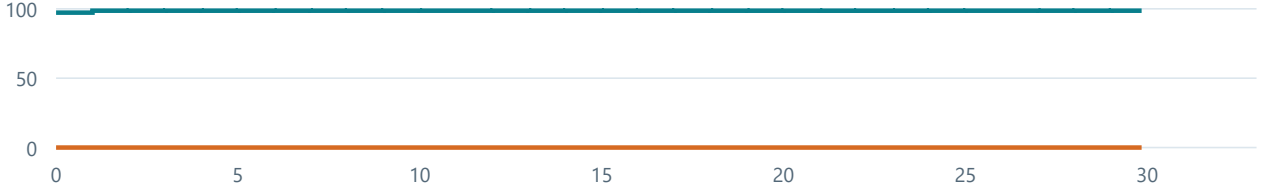
5 ms yük sırasında Idle tamamen ortadan kalkmıyor. Aktif pencerede yaklaşık %44,70 Idle bağlamı kalıyor; telemetri görevine atfedilen pay %52,95. Bu kayıt, yüksek öncelikli görevin tüm süre boyunca alt görevleri aç bıraktığını göstermiyor. Ancak her periyotta yaklaşık 5 ms'lik bir blok halinde çalışması, buton gibi daha düşük öncelikli görevlerde bekleme oluşturuyor.

Tam üçüncü bölümün Idle ortalaması %60,92. Testin aktif olmadığı baş/son süreleri çıkarıldığında %44,70'e düşüyor. Bu yüzden yalnızca tüm dosyanın CPU ortalamasına bakmak yükün etkisini olduğundan hafif gösterebilir.

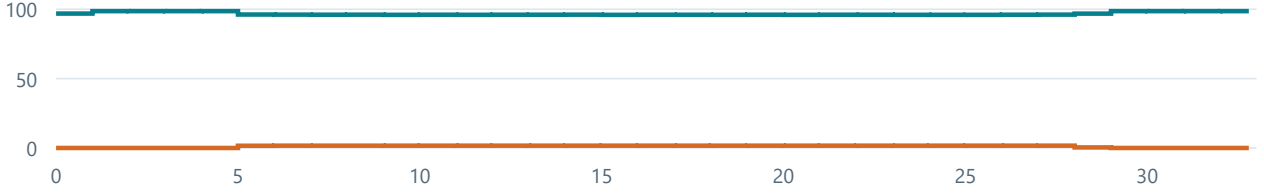
Yüzdelere nasıl okunmalı? Kaydedilen ISR süreleri task sürelerinden çıkarıldı. Fakat SysTick ile USART6/DMA taşıma kesmeleri ayrı olay olarak görünmüyor; bunların süresi o anda izlenen task/Idle bağlamının içinde kalabilir. Bu değerleri kesin donanım CPU kullanım yüzdesi olarak yorumlamayın.

Periyodik çalışma düzeni

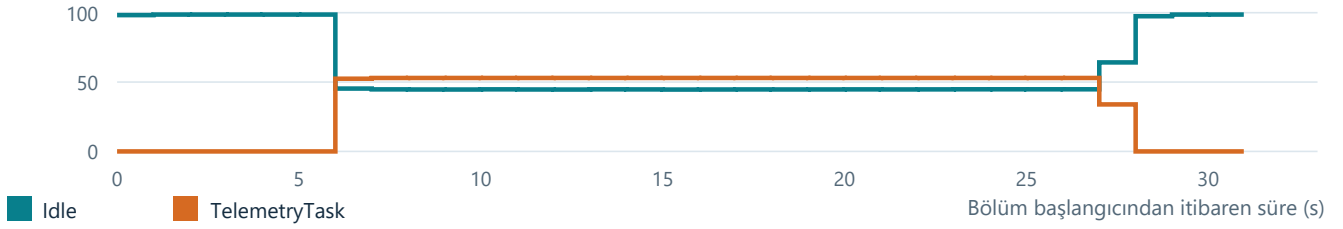
Telemetri kapalı



10 ms telemetri



10 ms telemetri + 5 ms yük



Grafik, bir saniyelik aralıklardaki bağlam paylarını gösterir. Ortadaki plato testin aktif olduğu süreyi, kenarlardaki Idle ağırlığı hazırlık/bitiş sürelerini gösterir. Kısa geçişleri bu grafik yerine olay zaman çizelgesiyle değerlendirmek gerekir.

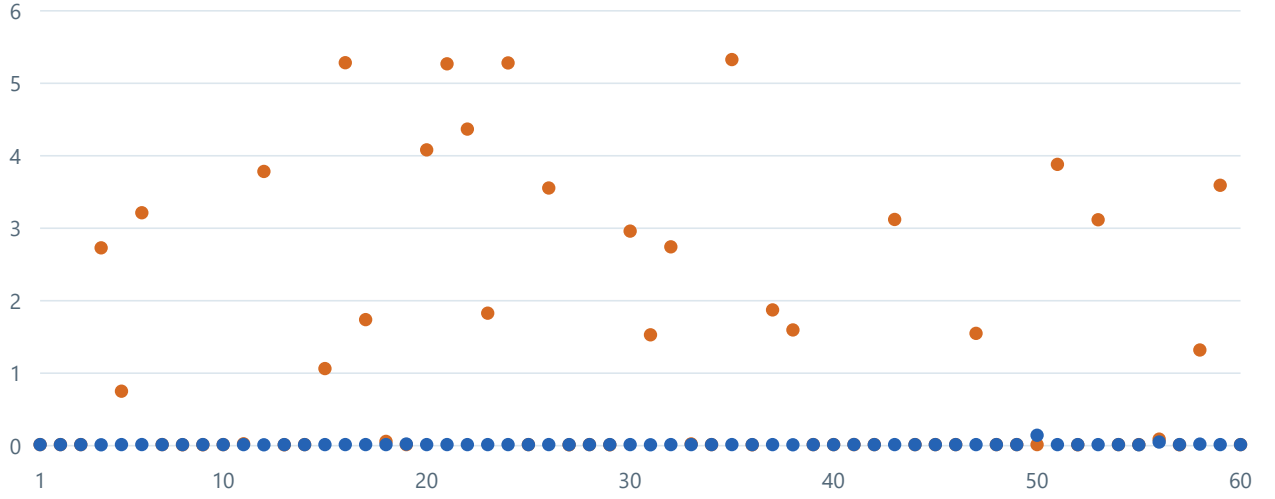
Telemetri göstergesi	10 ms telemetri	+ 5 ms yük
Kuyruk gönderim olayı	2.320	2.163
Aralık örnek sayısı	2.319	2.162
Ortalama üretim aralığı	10,000001 ms	10,000019 ms
En kısa / en uzun aralık	9,960 / 10,040 ms	9,939 / 10,062 ms
Task çalışma dilimi medyanı	0,167 ms	5,294 ms
Task çalışma dilimi en uzununu	0,171 ms	5,326 ms

10 ms üretim düzeni iki bölümde de korunuyor. Gözlenen kuyruk çağrısı aralıklarında kaçırılmış tam bir 10 ms periyoduna işaret eden boşluk yok. Üçüncü bölümde task çalışma dilimi medyanı yaklaşık 5,127 ms artıyor; bu artış dosya adındaki 5 ms yükü uyumlu.

Üretim aralığı, TelemetryTask içindeki kuyruk gönderim olayları arasındadır; UART hattındaki paket aralığı değildir. Task çalışma dilimi, task başlangıcından sonraki task/Idle geçişine kadar ona atfedilen süredir. Tam bir uygulama işiyle her zaman birebir eşleşmeyebilir; başlangıç/bitişte kısa dilimler vardır.

Buton gecikmesinin dağılımı

Bekleme (ms)



Her kaydın kendi içindeki buton örnek sayısı

Mavi: 10 ms telemetri. **Turuncu:** 10 ms telemetri + 5 ms yük. Her nokta bir ButtonTask READY/RUN eşleşmesidir. İki kayıttaki aynı sıra numarası, aynı fiziksel buton basışı anlamına gelmez.

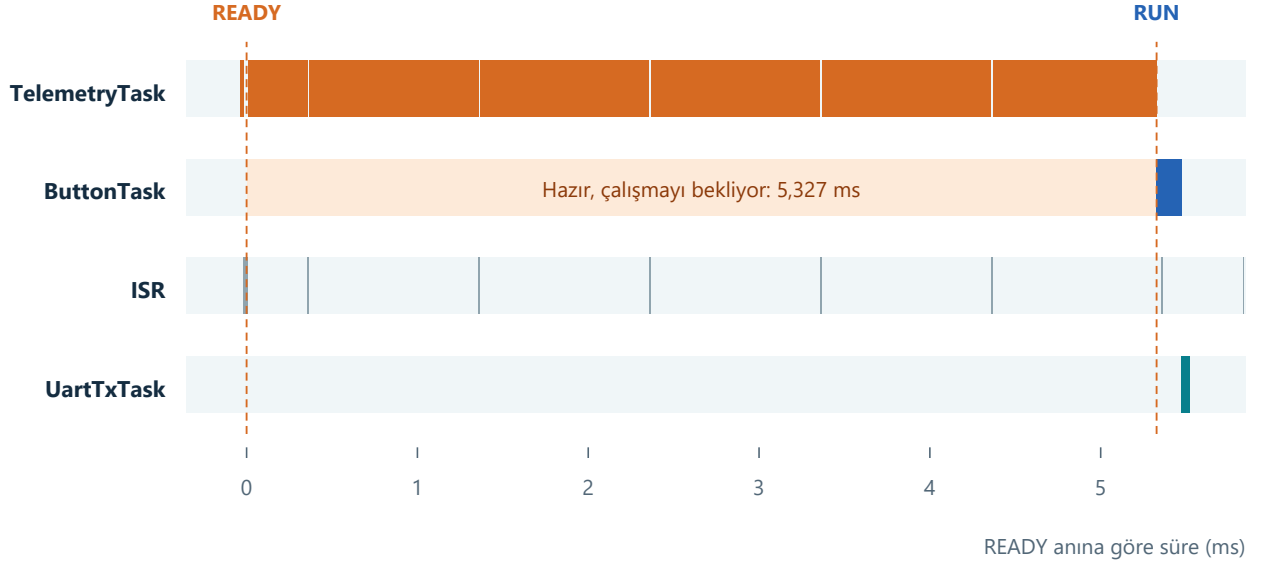
Bekleme göstergesi	10 ms telemetri	+ 5 ms yük
Örnek sayısı	60	60
Ortalama	0,0176 ms	1,2700 ms
Medyan	0,0148 ms	0,0150 ms
%95 yüzdeler (p95)	0,0215 ms	5,2693 ms
En uzun	0,1437 ms	5,3273 ms
1 ms'den uzun / 5 ms'den uzun	0 / 0	24 / 4

Medyanın aynı kalması, gecikme sorunu olmadığı anlamına gelmiyor. Yüklü durumda basışların bir kısmı telemetri uyurken geliyor ve hızla işleniyor. Telemetri çalışırken gelenler ise görevin kalan çalışma süresini bekliyor. Bu yüzden medyan yaklaşık 15 µs iken p95 yaklaşık 5,27 ms oluyor.

ButtonTask'ın kendi çalışma dilimi medyanı yüksüz/yüklü durumda sırasıyla 0,144 / 0,135 ms. Artan gecikmenin baskın nedeni buton yanıtını oluşturmanın yavaşlaması değil, görevin CPU sırası beklemesi. UartTxTask için de en uzun READY → RUN bekleme 0,183 ms'den 5,302 ms'ye yükseliyor.

Telemetri kapalı bölümde 50 EXTİ olayı var, ancak ButtonTask çalışma örneği yok. Mevcut Button_HandleEXTİ kodu g_running false iken olay oluşturmuyor; bu davranış kayıtlarla uyumlu. EXTİ sayısı fiziksel basış sayısı değildir; kenarlar ve debounce filtreleri farklı sayılar üretir.

En uzun beklemenin zaman çizelgesi



Bölüm içi zaman	Olay ve sonucu
15,854033 s	TelemetryTask çalışmaya başlıyor.
15,854071 s	Buton kesmesi sırasında ButtonTask READY oluyor.
15,854071 - 15,859398 s	ButtonTask hazır; daha yüksek öncelikli TelemetryTask çalışıyor.
15,859398 s	ButtonTask RUN oluyor. READY → RUN = 5,327319 ms.
15,859541 s	ButtonTask dilimi bitiyor; UartTxTask çalışmaya başlıyor.

Neden böyle oluyor?

Kayıt metadatasındaki öncelikler TelemetryTask=32, ButtonTask=24 ve UartTxTask=8. Bu sayılarda büyük değer daha yüksek task önceliği. Buton kesmesi telemetri çalışırken işlemciye girebiliyor ve buton görevini hazır yapıyor. Kesmenin işi bitince scheduler, hâlâ çalışabilir durumdaki daha yüksek öncelikli TelemetryTask'a dönüyor.

TelemetryTask kendi işini tamamlayıp beklemeye geçtiğinde ButtonTask çalışabiliyor. Bu, DMA'nın açık olmasıyla ortadan kalkmaz: DMA bayt taşımayı yapar; telemetri içindeki yapay hesaplama döngüsü CPU'da çalışmaya devam eder. DMA kullanılması task öncelik sırasını değiştirmez.

Bu örnek neyi kanıtlamıyor?

Çizelgede sonraki UartTxTask çalışması görünse de bu çalışmanın aynı butonun fiziksel UART bitişi olduğunu olay kimliği olmadan söyleyemeyiz. 20 ms uygulama hedefini değerlendirmek için aynı event_id'nin t0 ve t4 değerleri gerekir. Buradaki READY anı da uygulamadaki t0 veya t1 zaman damgasıyla birebir aynı nokta değildir.

Kanıt dosyası: veri/worst_button_events.csv. Genel events.csv içindeki ButtonTask başlangıç olayının sıfır tabanlı event_index değeri 241001. Çizelgede izlenen ISR aralıkları task şeritlerinden çıkarılmıştır.

UART kayıt trafiği ve kesmeler

Trace veri hızı	Telemetri kapalı	10 ms telemetri	+ 5 ms yük
Ortalama	10,17 kB/s	19,89 kB/s	19,71 kB/s
En yoğun tam 1 s	10,48 kB/s	24,15 kB/s	24,35 kB/s
921600 kapasitesi: ortalama	%11,04	%21,58	%21,39
921600 kapasitesi: en yoğun 1 s	%11,37	%26,20	%26,42
Overflow olayı	0	0	0

8N1 ayarında bir bayt 10 bit taşır: $921600 / 10 = \mathbf{92.160 \text{ bayt/s}}$. Tablo, kaydedilmiş SystemView ikili olay baytlarını bu kapasiteyle karşılaştırır; USB paketleri ve komut yönü dahil değildir. 5 ms hesaplama yükü CPU süresini artırmış, ancak trace veri hızını belirgin artırmamış. Hesaplama döngüsünün her iterasyonu ayrı bir trace olayı üretmiyor.

ISR giriş sayısı (tam bölüm)	Telemetri kapalı	10 ms telemetri	+ 5 ms yük
TIM6_DAC	29.841	32.788	30.973
USART1	0	2.446	2.289
DMA2_Stream7	0	4.884	4.570
EXTI15_10	50	122	120

TIM6 her üç bölümde yaklaşık 1 kHz. İzlenen toplam ISR payı tam bölümlerde %1,12 / %1,31 / %1,30. USART1 ve TX DMA kesmeleri aktif testlerde görünür; buton kesmeleri yük altında da işlenir. İzlenen en büyük ISR iç içe geçme derinliği telemetri açıkken 2'dir. Bu, tüm kesmelerin izlendiği anlamına gelmez.

defaultTask ve görünen periyodik uyanmalar

Üç bölümde de defaultTask çalışma olayı yok. Bu, mevcut main.c içindeki kendini askıya alan defaultTask ile uyumlu. Telemetri kapalıyken UartTxTask 1492 kez, yaklaşık 20 ms aralıklarla çalışıyor. Mevcut tek FIFO yolundaki osMessageQueueGet(..., 20u) zaman aşımı bunu açıklıyor; bu uyanma App_SystemView_Poll çağırısı değildir.

Overflow=0, uygulama kuyruklarında veya GUI paketlerinde kayıp=0 demek değildir. Başarılı ve başarısız kuyruk işlemlerinin bazı trace makroları aynı olay kimliğini kullanıyor. Kuyruk dönüş sonucu, uygulama drop sayaçları ve paket sıra numaraları bu raporda doğrulanmış değil.

Ölçüm sınırları ve stack bilgisi

Kaydın güçlü biçimde gösterdiği sonuçlar

10 ms telemetri üretim düzeni korunuyor. Yük sırasında TelemetryTask'ın çalışma dilimi uzuyor ve düşük öncelikli task'ların beklemesi artıyor. Yüklü bölümün aktif penceresinde Idle bağlamı var. İncelenen ikili akışta Overflow veya yapısal bozulma işareti yok.

Bu dosyalarla kesinleştirilemeyen noktalar

Toplam buton yanıtı: t0 → t4 ve GUI'ye varış için uygulama kayıtları gerekir. **Kesin CPU yükü:** SysTick ve USART6 taşıma kesmeleri ayrı görünmüyor. **Veri kaybı:** trace bütünlüğü, uygulama queue/drop veya UART paket başarısını tek başına kanıtlamaz. **Firmware eşleşmesi:** kaynakla ilişki kuruldu; kaydı üreten ELF'nin hash bilgisi dosyalarda bulunmadı.

Stack ekranındaki değeri dikkatli okumak gerekiyor

Mevcut SEGGER FreeRTOS adaptörü uxTaskGetStackHighWaterMark() değerini StackSize alanına koyuyor. Bu değer ayrılmış kapasite değil, en az boş kalmış stack alanının **32 bit sözcük** cinsinden değeri. Bu yüzden aşağıda 4 ile çarpıldı. StackUsage=0 alanı ise tek başına sıfır kullanım anlamına gelmez.

Task	Mevcut kodda kapasite	İlk snapshot boş alan
defaultTask	1024 B	203 sözcük = 812 B
UartTxTask	1536 B	198 sözcük = 792 B
ButtonTask	1024 B	170 sözcük = 680 B
TelemetryTask	1536 B	280 sözcük = 1120 B
Tmr Svc	1024 B	200 sözcük = 800 B

Her Start bölümünde beş stack kaydı var; değerler üçünde de aynı. Bunlar bölüm başlangıcındaki snapshot'lar. Sonraki ağır yükten sonra tekrar ölçülmediği için tüm test boyunca stack güvenliği veya RAM yeterliliği sonucu çıkarılamaz. Kapasite sütunu güncel kaynak ayarından alınmıştır.

Bir sonraki ölçümde anlamlı tamamlayıcı: Aynı senaryonun event_id, t0..t4 ve drop sayaçları ile bu READY → RUN dağılımını eşleştirmek. Böylece beklemenin toplam yanıt süresindeki payı ve varsa 20 ms hedef aşımı doğrudan görülebilir. Bu rapor kapsamında firmware değiştirilmedi.

Kanıtlar ve yeniden üretim

Dosya	İçerik
rapor.html	Bu raporun tek dosyalık, grafikler gömülü HTML sürümü.
veri/ozet.csv	Üç bağımsız bölüme ait temel karşılaştırma ölçümleri.
veri/events.csv	298.180 benzersiz olay; indeks, zaman, argümanlar.
veri/metrics.json	Bağlam aralıkları, READY örnekleri ve dağılımlar.
veri/decoded_summary.json	Dosya metadatası, görev bilgileri, SHA-256 değerleri.
veri/validation.json	Paket dönüşümü, bölüm toplamları, ISR denetimleri.
veri/worst_button_events.csv	En uzun ButtonTask beklemesinin olay kesiti.
veri/source_manifest.json	İnceleme sırasında kullanılan kaynakların hash değerleri.

Yeniden üretim sırası: **python analyze.py**, ardından **python validate.py**, ardından **python build_report.py**. İlk iki script Python standart kütüphanesiyle çalışır; çözücü kurulu SystemView FreeRTOS tanım dosyasını okur. Rapor oluşturucu ReportLab, pypdf ve Windows Segoe UI yazı tiplerini kullanır.

Hesaplama notları

Medyan ve yüzdelikler sıralanmış örneklerden hesaplandı; p95 için $(n-1) \times 0,95$ konumunda doğrusal enterpolasyon kullanıldı. En yoğun saniye, Start'a göre hizalı tam 1 s aralıklarının en büyüğüdür; kayan pencere veya anlık pik değildir. kB=1000 bayt. Süreler yuvarlandığı için tabloların son basamağında küçük toplam farkları olabilir.

Birincil kaynaklar

SEGGER SystemView User Guide: doc.segger.com/UM08027_SystemView.html. Olay/timestamp kavramları için başvuru; ikili paket yapısında yerel SEGGER_SYSVIEW.c esas alındı.

Yerel paket ve RTOS adaptörü: SEGGER_SYSVIEW.c::_SendPacket / _SendPacket_Ex; SEGGER_SYSVIEW.h olay kimlikleri; SEGGER_SYSVIEW_FreeRTOS.c::_cbSendTaskList / SYSVIEW_SendTaskInfo; SEGGER_SYSVIEW_FreeRTOS.h queue trace makroları.

Uygulama: Core/Src/app_tasks.c (UartTxTask_Run, ButtonTask_Run, TelemetryTask_Run), app_button.c (Button_HandleEXTI), app_systemview.c, stm32f7xx_it.c ve main.c. Öncelik/stack ayarları Core/Inc/app_config.h ve FreeRTOSConfig.h. Tüm yollar firmware/Button_Task altında; kaynak listesi manifestte tam olarak bulunur.

events.csv zamanı üç bölümün trace sürelerini art arda toplar; gerçek testler arasındaki duvar saati boşluğunu içermez. Rapordaki grafikler bölüm başlangıcına sıfırlanır. İlk dosyanın olay aralığı 0..67258, ikinci yeni bölüm 67259..186255, üçüncü yeni bölüm 186256..298179 (sıfır tabanlı).